



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA
IN INGEGNERIA INFORMATICA

DOCUMENTO TECNICO: BRIDGE TRA MAGIC LEAP 2 E ROS 2 ED ESTENSIONE A COMUNICAZIONE BIDIREZIONALE

Relatore:

Prof. Stefano Ghidoni

Correlatore:

Matteo Terreran, PhD

Candidati:

Federico Compagno

Matricola: 2101752

Alessandro Rubert

Matricola: 2101793

Anno Accademico 2025/2026

Sommario

Nel contesto dell'Industria 4.0 e dell'evoluzione verso la *Human-Robot Collaboration* (HRC), la rimozione delle barriere fisiche tra operatori umani e manipolatori industriali richiede lo sviluppo di interfacce di interazione naturali, intuitive e a sicurezza intrinseca. I sistemi tradizionali basati su monitor bidimensionali o pulsantiere fisse vincolano l'utente e limitano la percezione spaziale congiunta; la Realtà Aumentata (AR) si pone dunque come tecnologia abilitante per colmare questo divario, sovrapponendo elementi informativi e olografici direttamente nel campo visivo dell'operatore.

Questo lavoro di tesi descrive la progettazione, lo sviluppo e la validazione sperimentale di un'infrastruttura software modulare ad alte prestazioni per la comunicazione bidirezionale in tempo reale tra il visore di realtà aumentata **Magic Leap 2** e il middleware robotico **ROS 2**. L'architettura sviluppata si articola in un'applicazione grafica implementata in ambiente Unity e un ponte di comunicazione di rete basato sul protocollo **ROS-TCP-Connector**, che garantisce la serializzazione asincrona e il trasferimento dati a bassa latenza tra l'headset e i nodi ROS 2. Il modulo consente lo streaming continuo di flussi video ad alta risoluzione (Picture Camera, World Cameras, Depth Camera) e dati sensoristici inerziali (IMU), integrando inoltre l'acquisizione automatica dei parametri intrinseci di calibrazione ottica pubblicati sul topic standard `sensor_msgs/msg/CameraInfo`. Il sistema implementa una pipeline di configurazione dinamica disaccoppiata basata su file JSON, consentendo l'abilitazione e la personalizzazione dei singoli flussi a runtime senza necessità di ricompilare il progetto.

Per garantire la coerenza geometrica nello spazio operativo, la localizzazione globale e l'allineamento tra il sistema di riferimento del visore (`magicleap_world`) e la base del manipolatore sono ottenuti mediante il tracciamento ottico di un marcatore **AprilTag** e la gestione dinamica dell'albero delle trasformazioni spaziali (TF2). Sfruttando tale ancoraggio, è stato sviluppato un modulo di interazione 3D in cui l'operatore, mediante tecniche di *raycasting* dal controller a 6 gradi di libertà, seleziona una posa target su un piano di lavoro virtuale con riscontro visivo olografico istantaneo. La coordinata viene trasmessa al robot tramite il topic `/target_ball_pose`; mediante il framework **MoveIt 2**, il robot collaborativo **Franka Emika Panda** calcola la cinematica inversa ed esegue la traiettoria priva di collisioni in modo automatico e sicuro.

I risultati sperimentali dimostrano l'efficacia e la robustezza dell'infrastruttura, evidenziando un'elevata fluidità di trasmissione nei flussi video multi-camera, l'assenza di drift negli ancoraggi virtuali e un'esperienza di controllo robotico naturale ed immediata.

Panoramica del Progetto: Puoi trovare la struttura completa del codice e la documentazione aggiuntiva clonando o visitando il repository qui.

Indice

Sommario	2
Indice	4
1 Introduzione e Contesto	6
1.1 Contesto e Motivazione: Human-Robot Collaboration (HRC)	6
1.2 La Realtà Aumentata (AR) e lo Spatial Computing	6
1.3 Obiettivo del Lavoro e del Modulo Software	7
1.4 Stack Tecnologico e Requisiti di Sistema	8
2 Progettazione e Implementazione del Bridge	9
2.1 Requisiti Funzionali e Vincoli	9
2.1.1 Requisiti Funzionali	9
2.1.2 Vincoli di Progetto e di Sistema	10
2.2 Architettura del Bridge e Negoziazione dei Parametri	10
2.2.1 Componenti del Sistema e Flusso di Negoziazione	10
2.2.2 Struttura del File di Configurazione (config.yaml)	11
2.2.3 Architettura a Componenti e Modello di Esecuzione in Unity	13
2.3 Implementazione e Caratterizzazione dei Moduli Sensoriali	13
2.3.1 Telecamere di Tracciamento Ambientale (World Cameras)	13
2.3.2 Telecamera Frontale RGB ad Alta Risoluzione (Picture Center)	15
2.3.3 Sensore di Profondità (Depth camera)	17
2.3.4 Telecamere per l'Eye Tracking e Stima dello Sguardo	19
2.3.5 Unità Inerziali (IMU) e Sensore di Luminosità Ambientale	21
2.4 Manuale D'uso	21
2.4.1 Configurazione dell'Ambiente ROS 2	22
2.4.2 Installazione e Configurazione dell'Applicazione su Magic Leap 2	22
2.4.3 Sequenza di Avvio e Streaming dei Dati	22
2.5 Modularità e Upgrade	23
2.5.1 Setup dell'Ambiente di Sviluppo Unity	23
2.5.2 Linee Guida per l'Estensione e l'Upgrade del Codice	23
3 Estensione del Bridge: Robot Interaction	24
3.1 Obiettivo e Architettura dell'Applicazione di Interazione	24
3.2 Localizzazione Spaziale e Calibrazione tramite AprilTag	25
3.2.1 Albero delle Trasformazioni Spaziali (TF Tree)	25
3.2.2 Composizione delle Trasformazioni	25
3.3 Interazione 3D e Definizione del Target in Realtà Aumentata	27

3.3.1	Piano di Lavoro Virtuale e Raycasting da Controller	27
3.3.2	Struttura del Messaggio di Comunicazione	27
3.4	Pianificazione del Moto ed Esecuzione con MoveIt 2	28
3.4.1	Conversione delle Coordinate nel Frame Base del Robot	28
3.4.2	Cinematica Inversa e Generazione delle Traiettorie	28
3.5	Procedura di Avvio e Configurazione Sperimentale	29
3.6	Fasi di Esecuzione	32
3.6.1	Puntamento 3D e Selezione del Target	32
3.6.2	Pianificazione ed Esecuzione della Traiettoria	32
3.6.3	Ritorno in Configurazione di Sicurezza	33
3.7	Validazione Sperimentale e Risultati	33

Capitolo 1

Introduzione e Contesto

1.1 Contesto e Motivazione: Human-Robot Collaboration (HRC)

Negli ultimi anni, l'evoluzione del panorama industriale verso il paradigma di **Industria 4.0** e la successiva transizione verso l'—ancora più centrata sull'uomo— **Industria 5.0** hanno trasformato radicalmente il concetto di automazione. I robot industriali tradizionali, caratterizzati da elevata velocità, rigidità e potenza, richiedono da sempre di operare all'interno di gabbie di protezione segregate per prevenire collisioni e tutelare l'incolumità fisica degli operatori umani.

L'avvento della *Human-Robot Collaboration* (HRC) e l'introduzione dei robot collaborativi (*cobot*) hanno abbattuto queste barriere fisiche, consentendo all'operatore e al braccio robotico di condividere lo stesso spazio di lavoro (*shared workspace*). I cobot, dotati di sensori di coppia ai giunti e logiche di arresto di sicurezza per contatto, uniscono due mondi complementari:

- **L'essere umano:** porta flessibilità, capacità decisionale improvvisata, adattabilità sensoriale e percezione tattile complessa;
- **Il robot:** offre precisione millimetrica ripetibile, capacità di sollevamento carichi senza affaticamento ed esecuzione costante di compiti ripetitivi.

Tuttavia, affinché una cella collaborativa risulti veramente efficace e sicura, non è sufficiente garantire l'assenza di lesioni in caso di contatto accidentalmente avvenuto: è necessario **anticipare e coordinare le azioni**. I canali di comunicazione tradizionali basati su monitor 2D, tastiere o *teach pendant* rigidi introducono un elevato carico cognitivo (*cognitive load*), costringendo l'operatore a distogliere continuamente lo sguardo dal campo di lavoro fisico per consultare l'interfaccia grafica.

1.2 La Realtà Aumentata (AR) e lo Spatial Computing

La Realtà Aumentata (AR) si propone come tecnologia abilitante fondamentale per superare i limiti delle interfacce bidimensionali. Sovrapponendo informazioni digitali e olografiche tridimensionali direttamente nel campo visivo naturale dell'operatore, l'AR consente di:

1. **Visualizzare le intenzioni del robot:** proiettare traiettorie pianificate, punti di destinazione (*target pose*) o volumi di ingombro futuro prima che il manipolatore inizi il movimento fisico;
2. **Fornire comandi spaziali intuitivi:** consentire all'operatore di interagire direttamente con lo spazio tridimensionale reale mediante puntamento con controller 6-DoF, gesto manuale o tracciamento dello sguardo (*eye tracking*);
3. **Mantenere l'operatività hands-free e in contesto:** eliminare la necessità di postazioni fisse, permettendo all'utente di muoversi liberamente attorno alla cella di lavoro.

L'utilizzo di un visore AR avanzato come il Magic Leap 2 permette di estrarre e condividere un'enorme quantità di flussi sensoriali (camere RGB, telecamere di tracciamento ambiente, mappe di profondità, IMU e parametri di calibrazione ottica). Integrare questi flussi in un ecosistema robotico distribuito come **ROS 2** apre prospettive avanzate non solo per la teleoperazione, ma anche per il monitoraggio sensoriale e la percezione remota.

1.3 Obiettivo del Lavoro e del Modulo Software

Il presente lavoro nasce dall'esigenza di realizzare un'infrastruttura software modulare, robusta e a bassa latenza che colleghi in modo **bidirezionale** il visore AR Magic Leap 2 con il sistema operativo robotico ROS 2.

In particolare, gli obiettivi principali previsti e realizzati nel modulo riguardano:

- **Streaming bidirezionale ad alte prestazioni:** trasmissione continua dal visore verso ROS 2 di flussi video ad alta risoluzione (RGB, World, Depth) e dati sensoristici (IMU e info di calibrazione ottica **CameraInfo**);
- **Disaccoppiamento e configurazione dinamica a runtime:** sviluppo di una pipeline basata su file di configurazione in formato JSON che permette di abilitare, disabilitare o riconfigurare framerate e risoluzione dei sensori del visore senza dover ricompilare il progetto in Unity;
- **Localizzazione e allineamento spaziale (TF2):** ancoraggio stabile tra il sistema di riferimento del visore (**magicleap_world**) e la base del robot tramite tracciamento ottico di marcatori AprilTag;
- **Interazione 3D e controllo del manipolatore:** sviluppo di una logica di puntamento tridimensionale (raycasting con controller ML2) per la selezione di punti target nello spazio AR, con invio immediato delle coordinate a MoveIt 2 per la pianificazione di traiettorie prive di collisioni sul cobot Franka Emika Panda.

L'output del lavoro non si limita alla convalida sperimentale, ma fornisce un modulo software documentato e riutilizzabile, pensato per essere facilmente integrato, modificato o esteso all'interno del laboratorio per futuri progetti di ricerca.

1.4 Stack Tecnologico e Requisiti di Sistema

Nella Tabella 1.1 vengono riassunti i componenti hardware e software fondamentali che costituiscono la piattaforma di sviluppo e sperimentazione.

Tabella 1.1: Componenti Hardware e Software del Setup Sperimentale.

Ambito	Componente	Descrizione / Versione
Hardware	Magic Leap 2	Visore AR con Compute Pack (CPU Zen 2, GPU RDNA 2, 16GB RAM) e Controller 6-DoF.
	Franka Emika Panda	Cobot collaborativo a 7 gradi di libertà con sensori di coppia ai giunti.
	Router di Rete	Router/Switch Gigabit per la rete locale TCP/IP.
Software	ROS 2	Framework robotico distribuito (versione Humble / Iron).
	Unity	Engine 3D (LTS 2022.3.62f3) per lo sviluppo dell'applicazione AR.
	Magic Leap SDK	SDK dedicato per l'accesso alle API dei sensori e al rendering AR.
	ROS-TCP-Connector	Middleware socket TCP per la comunicazione bidirezionale Unity ↔ ROS 2.
	MoveIt 2	Framework di motion planning e cinematica inversa.
	AprilTag ROS	Nodo per il tracciamento ottico di marcatori di calibrazione.

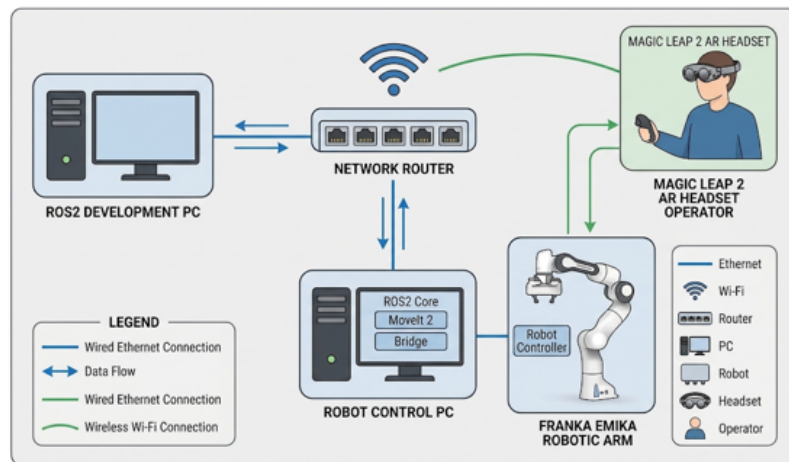


Figura 1.1: Architettura hardware e di rete del setup sperimentale.

Capitolo 2

Progettazione e Implementazione del Bridge

2.1 Requisiti Funzionali e Vincoli

La progettazione dell'architettura di comunicazione tra il visore Magic Leap 2 e l'ecosistema ROS 2 ha richiesto un'attenta analisi preliminare sia degli obiettivi applicativi, sia delle limitazioni hardware e computazionali tipiche dei dispositivi embedded e della trasmissione wireless. Di seguito vengono formalizzati i requisiti funzionali individuati e i vincoli che hanno guidato lo sviluppo del software.

2.1.1 Requisiti Funzionali

I requisiti funzionali definiscono le capacità operative e i servizi che il bridge software sviluppato a bordo del visore deve garantire:

- **Acquisizione Multi-Sensoriale:** Il sistema deve interfacciarsi con l'hardware del visore per estrarre flussi eterogenei di dati. Nello specifico, deve acquisire i flussi video (RGB e telecamere di tracking ambientale), la stima della posa nello spazio 3D calcolata dall'headset e le letture dei sensori ausiliari (come IMU e sensore di luminosità ambientale).
- **Serializzazione e Trasmissione Dati su ROS 2:** Le informazioni sensoriali estratte devono essere impacchettate e serializzate rispettando i messaggi standard di ROS 2 (in particolare la suite `sensor_msgs` e i relativi tipi per immagini, pose e letture grezze). Per ogni flusso video, il bridge deve garantire la pubblicazione sincronizzata del relativo messaggio di calibrazione intrinseca (`CameraInfo`), permettendo la corretta interpretazione ottica delle immagini da parte dei nodi riceventi.
- **Configurazione Dinamica a Runtime:** Per massimizzare la flessibilità operativa ed evitare continui cicli di ricompilazione del package Unity/Android su visore, il sistema deve consentire la configurazione dei parametri di streaming (quali sensori abilitare, risoluzioni e framerate desiderati) direttamente dal lato ROS 2 tramite un file di configurazione letto all'avvio dell'applicazione.

2.1.2 Vincoli di Progetto e di Sistema

I vincoli architetturali e tecnologici hanno indirizzato le scelte implementative per garantire massima compatibilità, efficienza e scalabilità:

- **Standardizzazione e Interoperabilità:** Per consentire l'integrazione immediata dei dati del visore con algoritmi terzi della comunità robotica (come pipeline di visual SLAM o nodi di controllo e pianificazione), l'architettura dei topic e l'albero delle trasformazioni geometriche (`tf`) sono stati allineati agli standard de facto utilizzati dai principali sensori di visione robotica. Ciò ha comportato l'adozione rigorosa delle convenzioni di orientamento per i frame ottici (`_optical_frame`), garantendo che il sistema di riferimento delle telecamere rispetti la convenzione ottica standard e una nomenclatura dei topic standardizzata.
- **Efficienza di Rete e Gestione delle Risorse:** Lo streaming di molteplici flussi sensoriali e di frame video ad alta risoluzione tramite rete introduce un potenziale collo di bottiglia sulla banda disponibile. Il sistema è stato vincolato all'ottimizzazione del payload trasmesso e alla minimizzazione dell'overhead di serializzazione del protocollo TCP, garantendo frequenze di aggiornamento adeguate senza saturare la rete locale o sovraccaricare la CPU del visore.
- **Modularità ed Estensibilità del Software:** L'applicazione su Unity è stata concepita seguendo un pattern a componenti disaccoppiati (script indipendenti per ciascuna tipologia di sensore). Questo vincolo progettuale assicura la manutenibilità del codice e consente di integrare agevolmente in futuro nuovi flussi sensoriali o logiche di controllo robotico senza dover riscrivere la logica di connessione o interferire con i moduli già collaudati.

2.2 Architettura del Bridge e Negoziazione dei Parametri

Per garantire un disaccoppiamento efficace tra la logica di controllo su ROS 2 e l'esecuzione a bordo del visore, il sistema è stato progettato secondo un modello modulare che inizia con una fase iniziale di negoziazione e configurazione dinamica dei parametri di trasmissione. L'infrastruttura si compone di due nodi sul lato ROS 2 (l'endpoint di comunicazione e il nodo di configurazione) e di un insieme coordinato di script a bordo dell'applicazione Unity.

2.2.1 Componenti del Sistema e Flusso di Negoziazione

Il funzionamento del bridge si basa su un meccanismo di sincronizzazione a stati tra ROS 2 e Unity:

1. **Avvio dell'Infrastruttura ROS 2:** Vengono inizializzati l'endpoint di rete (`ROS-TCP-Endpoint`), incaricato di gestire la connessione a basso livello con Unity, e il nodo di configurazione. Quest'ultimo carica in memoria i parametri operativi definiti all'interno di un file `config.yaml` e si mette in ascolto sul topic dedicato alle richieste provenienti dal visore.

2. **Richiesta di Configurazione (Handshake):** All'avvio dell'applicazione su Magic Leap 2, lo script coordinatore (**Configurator**) invia un messaggio di sincronizzazione sul topic di richiesta (**/request**).
3. **Serializzazione e Invio dei Parametri:** Ricevuta la richiesta, il nodo di configurazione serializza i parametri letti dal file YAML in una stringa strutturata (formato JSON) e la pubblica su un apposito topic di risposta destinato al visore (**/magic_leap2/config**).
4. **Parsing e Avvio Selettivo dei Flussi:** Il **Configurator** a bordo di Unity riceve il payload JSON e ne effettua il parsing. La struttura modulare dell'applicazione prevede uno script per la gestione di ogni sensore o funzionalità. Questi script partono a priori tutti insieme ma, prima di avviare la lettura e l'invio dei messaggi a ROS 2, attendono che il **Configurator** assegni loro i rispettivi parametri (abilitazione, risoluzioni, framerate).
5. **Attivazione dei Sensori:** Completata l'assegnazione dei parametri, il **Configurator** imposta a **true** i flag di avvio (**enabled**) dei soli moduli attivati. Tale transizione sblocca l'esecuzione delle coroutine di streaming all'interno dei rispettivi script, dando inizio alla pubblicazione continua dei dati sensoriali.

2.2.2 Struttura del File di Configurazione (config.yaml)

La configurazione centralizzata consente all'utente di definire in anticipo il carico computazionale e l'occupazione di banda della rete senza dover modificare o ricompilare il codice su Unity o Android. Tramite il file **config.yaml** è possibile specificare il prefisso dei topic, l'abilitazione selettiva dei sensori, le modalità di stream, le risoluzioni e le frequenze di pubblicazione.

Di seguito viene riportata la struttura del file **config.yaml** adottato:

```
/config_ML_node:
  ros__parameters:
    general:
      topic_name: "/magic_leap2"
    picture_center:
      enabled: false
      stream: 0
      resolution_width: 1920
      resolution_height: 1080
      update_rate: 30
      format: "jpeg"
    depth_center:
      enabled: false
      stream: 1
      resolution_width: 544
      resolution_height: 480
      update_rate: 30
      format: "Depth32"
    world_center:
      enabled: false
```

```
stream: 0
resolution_width: 1016
resolution_height: 1016
update_rate: 30
format: "Grayscale"
world_right:
  enabled: false
  stream: 0
  resolution_width: 1016
  resolution_height: 1016
  update_rate: 30
  format: "Grayscale"
world_left:
  enabled: false
  stream: 0
  resolution_width: 1016
  resolution_height: 1016
  update_rate: 30
  format: "Grayscale"
eye_temple_right:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"
eye_temple_left:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"
eye_nasal_right:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"
eye_nasal_left:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"
other_sensors:
```

`enabled: false`

2.2.3 Architettura a Componenti e Modello di Esecuzione in Unity

All'interno dell'ambiente Unity, l'applicazione è strutturata attorno a un unico `GameObject` centrale a cui sono associati script `C#` dedicati e indipendenti per ciascuna tipologia di flusso sensoriale. Ciascun modulo implementa un modello di esecuzione comune basato sulle seguenti fasi:

- **Inizializzazione (Start):** Configura le strutture dati interne e i publisher di ROS 2, mantenendo una variabile booleana di abilitazione inizialmente impostata a `false`.
- **Sblocco e Controllo del Flusso (Update):** A ogni frame di rendering, il metodo `Update` monitora lo stato del flag di abilitazione. Non appena lo script di configurazione imposta il flag a `true`, viene avviata una coroutine asincrona dedicata all'acquisizione dei frame e alla pubblicazione dei messaggi `sensor_msgs/Image` e `sensor_msgs/CameraInfo` con il relativo timestamp.
- **Aggiornamento Continuo delle Pose:** Parallelamente all'invio dei frame, il ciclo `Update` continua a estrarre la posa 6-DoF aggiornata del sensore (espressa rispetto al frame ottico di riferimento) e a pubblicarla su ROS 2. Questa separazione consente di mantenere l'albero delle trasformazioni geometriche (`tf`) aggiornato ad alta frequenza, senza vincolarlo ai frame acquisiti dalle telecamere.
- **Integrazione di Sensori a Basso Livello:** Per i flussi non gestiti nativamente tramite le API standard di Unity (come i sensori IMU e di luminosità), lo script dedicato interroga a runtime un plugin nativo sviluppato in Kotlin per Android OS, esponendo le letture verso l'infrastruttura di pubblicazione.

2.3 Implementazione e Caratterizzazione dei Moduli Sensoriali

2.3.1 Telecamere di Tracciamento Ambientale (World Cameras)

Descrizione e Funzionamento Il visore integra tre telecamere monocromatiche ad ampio campo visivo (FOV), ottimizzate a bassa latenza per l'analisi spaziale e la comprensione dell'ambiente circostante. I sensori sono disposti lungo il telaio e identificati convenzionalmente come *World Center*, *World Left* e *World Right*. Ciascun sensore acquisisce immagini in scala di grigi con una risoluzione fissa di 1016×1016 pixel a 30 Hz.

Topic ROS 2 e Messaggi La struttura dei topic per una generica camera `<name>` (con `<name> ∈ {world_center, world_left, world_right}`) è riportata nella Tabella 2.1:

Tabella 2.1: Topic ROS 2 per le Telecamere di Tracciamento Ambientale (World Cameras).

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/<name>/image_raw	sensor_msgs/msg/Image	Flusso grezzo delle immagini acquisite.
/magic_leap2/<name>/camera_info	sensor_msgs/msg/CameraInfo	Parametri intrinseci e modello di calibrazione della telecamera.
/magic_leap2/<name>/metadata	std_msgs/msg/String	Metadati del sensore (es. guadagno analogico/digitale, tempo di esposizione).



Figura 2.1: Immagine estratta dallo stream per la World Center Camera

Trasformazioni Spaziali (TF Tree) Il bridge pubblica le pose delle telecamere sul topic standard `/tf` a 60 Hz rispetto a `magic Leap world`. Vengono pubblicati due specifici frame per sensore:

- `<name>_frame`: orientato secondo la convenzione robotica standard (X avanti, Y sinistra, Z in alto);
- `<name>_optical_frame`: orientato secondo la convenzione ottica standard (Z avanti lungo l'asse ottico, X a destra, Y in basso).

Limitazioni ed Eventuali Criticità Trattandosi di stream non compressi ad alta risoluzione (1016×1016 pixel a 8 bit), l'attivazione contemporanea di tutti e

tre i moduli genera un carico di rete complessivo di circa 750 Mbit/s. Tale volume può saturare reti Wi-Fi convenzionali, rendendo consigliabile l'abilitazione selettiva della sola telecamera di interesse o l'uso di una connessione cablata.

2.3.2 Telecamera Frontale RGB ad Alta Risoluzione (Picture Center)

Descrizione e Funzionamento Posizionata frontalmente al centro del telaio, la telecamera *Picture Center* è concepita per l'acquisizione di dettagli visivi e registrazioni ad alta definizione. Il flusso converte e comprime nativamente i frame in formato JPEG direttamente a bordo del dispositivo. Le API mettono a disposizione due modalità:

- **Stream 0 (RGB Standard Stream):** Flusso video ottico convenzionale della camera RGB.
- **Stream 1 (Mixed Reality / AR Stream):** Flusso composito in cui al video reale dell'ambiente circostante vengono sovrapposti gli elementi grafici e gli ologrammi visualizzati dall'utente nel visore.

Topic ROS 2 e Messaggi In base alla modalità selezionata nel file `config.yaml`, il bridge pubblica i dati sui topic descritti nelle Tabelle 2.2 e 2.3:

Tabella 2.2: Topic ROS 2 per la RGB Camera (Stream 0).

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/color/image_raw/compressed	sensor_msgs/msg/CompressedImage	Flusso delle immagini compresse acquisite.
/magic_leap2/color/camera_info	sensor_msgs/msg/CameraInfo	Parametri intrinseci e modello di calibrazione della telecamera.
/magic_leap2/color/metadata	std_msgs/msg/String	Metadati del sensore (es. guadagno analogico/digitale, tempo di esposizione).

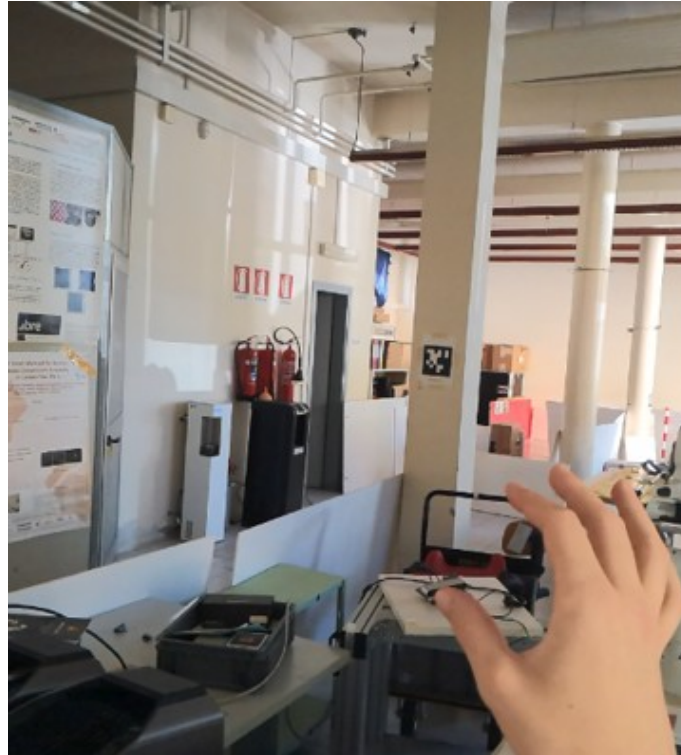


Figura 2.2: Immagine estratta dallo stream per la RGB Camera

Tabella 2.3: Topic ROS 2 per la modalità Mixed Reality (Stream 1).

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/mixed_reality/ image_raw/compressed	sensor_msgs/ msg/CompressedImage	Flusso delle immagini compresse acquisite.
/magic_leap2/mixed_reality/ camera_info	sensor_msgs/ msg/CameraInfo	Parametri intrinseci e modello di calibrazione.
/magic_leap2/mixed_reality/ metadata	std_msgs/ msg/String	Metadati del sensore.

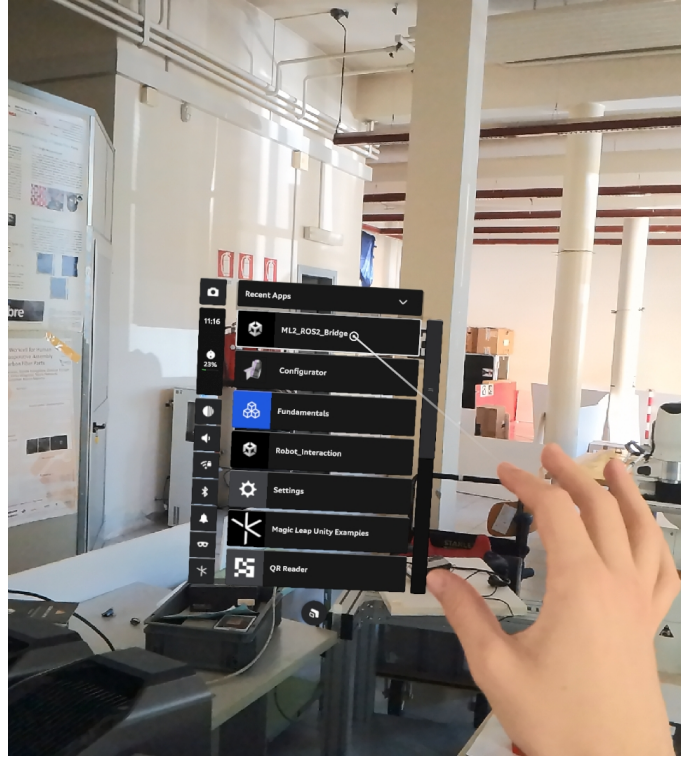


Figura 2.3: Immagine estratta dallo stream per la Mixed Reality

Trasformazioni Spaziali (TF Tree) Il bridge pubblica le pose sul topic `/tf` a 60 Hz rispetto a `magicleap_world`, generando i frame `color_frame` e `color_optical_frame`.

Limitazioni ed Eventuali Criticità L'acquisizione ad alta risoluzione combinata con la compressione JPEG continua genera un rapido incremento della temperatura del visore, inducendo fenomeni di *thermal throttling* che possono degradare progressivamente il framerate effettivo nell'arco di una decina di minuti.

2.3.3 Sensore di Profondità (Depth camera)

Descrizione e Funzionamento Posizionato frontalmente (*Depth Center*), il sensore si basa su tecnologia a tempo di volo (Time-of-Flight, ToF) e opera a una risoluzione fissa di 544×480 pixel fornendo due formati di output:

- **Depth32 (Mappa di Profondità):** Matrice di valori a 32 bit (float32) in cui ciascun elemento rappresenta la distanza metrica stimata.
- **DepthRaw (Intensità Infrarossa):** Matrice di valori a 32 bit (float32) contenente la quantità grezza di luce infrarossa riflessa.

Le API supportano lo *Stream 0* (Long Range, 0.5–5.0 m a 1 o 5 Hz) e lo *Stream 1* (Short Range, 0.2–1.2 m a 5 o 30 Hz).

Topic ROS 2 e Messaggi A seconda del formato selezionato, i dati vengono pubblicati sui topic indicati nelle Tabelle 2.4 e 2.5:

Tabella 2.4: Topic ROS 2 per il formato Depth32.

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/depth/image_raw	sensor_msgs/msg/Image	Matrice grezza di profondità metrica (codifica 32FC1).
/magic_leap2/depth/camera_info	sensor_msgs/msg/CameraInfo	Parametri intrinseci e calibrazione ToF.
/magic_leap2/depth/metadata	std_msgs/msg/String	Metadati operativi del sensore ToF.



Figura 2.4: Immagine estratta dallo stream per Depth Camera - formato Depth32 (long range)

Tabella 2.5: Topic ROS 2 per il formato DepthRaw.

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/infra/image_raw	sensor_msgs/msg/Image	Matrice grezza intensità riflessione IR (32FC1).
/magic_leap2/infra/camera_info	sensor_msgs/msg/CameraInfo	Parametri intrinseci del sensore IR.
/magic_leap2/infra/metadata	std_msgs/msg/String	Metadati del ricevitore IR.



Figura 2.5: Immagine estratta dallo stream per la Depth Camera - formato Depth-Raw (long range)

Trasformazioni Spaziali (TF Tree) Il bridge pubblica le pose a 60 Hz sul topic `/tf` rispetto a `magic Leap world`, associandole ai frame `depth_frame` e `depth_optical_frame`.

Limitazioni ed Eventuali Criticità Nella modalità Short Range (Stream 1), sebbene l'API consenta nominalmente la selezione di un framerate a 60 Hz, la frequenza effettiva di pubblicazione rimane vincolata a un limite hardware/firmware massimo di 30 Hz.

2.3.4 Telecamere per l'Eye Tracking e Stima dello Sguardo

Descrizione e Funzionamento Il visore integra quattro telecamere monocromatiche a infrarossi per il tracciamento oculare, posizionate a coppie sul lato nasale e temporale di ciascun occhio (`eye_nasal_right`, `eye_temple_right`, `eye_nasal_left`, `eye_temple_left`). Ciascuna camera acquisisce immagini a 400×400 pixel in scala di grigi a 30 Hz.

Topic ROS 2 e Messaggi I flussi attivati vengono pubblicati sui topic riportati nella Tabella 2.6:

Tabella 2.6: Topic ROS 2 per le Telecamere di Eye Tracking.

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/<name>/image_raw</code>	<code>sensor_msgs/msg/Image</code>	Flusso grezzo delle immagini monocromatiche (mono8).
<code>/magic_leap2/<name>/metadata</code>	<code>std_msgs/msg/String</code>	Metadati del sensore oculare.

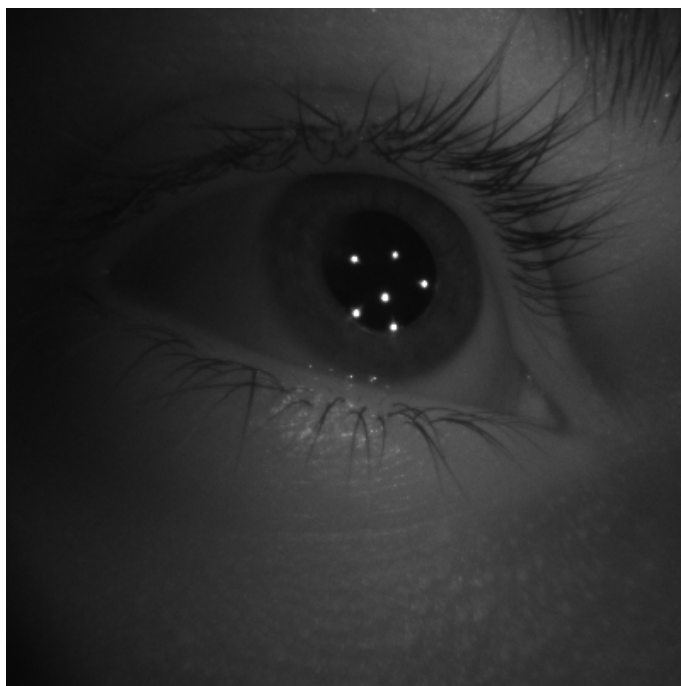


Figura 2.6: Immagine estratta dallo stream per la Eye Temple Left Camera

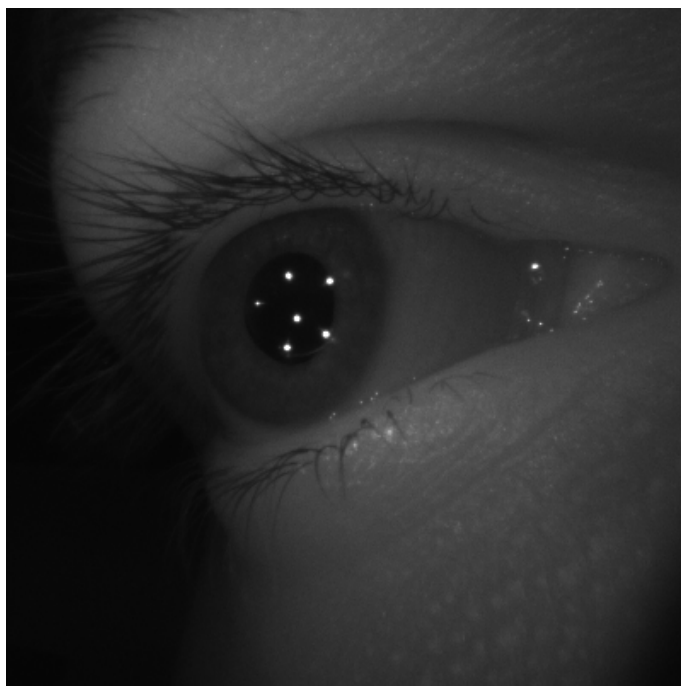


Figura 2.7: Immagine estratta dallo stream per la Eye Nasal Left Camera

Limitazioni ed Eventuali Criticità Le API di Magic Leap non espongono la matrice di calibrazione intrinseca né i coefficienti di distorsione per le camere oculari, impedendo la generazione del topic `sensor_msgs/msg/CameraInfo`. Inoltre, non vengono fornite le trasformazioni spaziali rigide rispetto al telaio centrale del visore, motivo per cui non sono pubblicati frame dedicati sull'albero `/tf`.

2.3.5 Unità Inerziali (IMU) e Sensore di Luminosità Ambientale

Descrizione e Funzionamento L'architettura hardware comprende tre unità IMU distinte (due sull'headset, sinistra e destra, e una all'interno del Compute Pack) oltre a un sensore di luminosità ambientale. Per superare i limiti dell'SDK standard, l'acquisizione è delegata a un plugin nativo Android sviluppato in Kotlin che interroga direttamente il `SensorManager` a basso livello, garantendo timestamp hardware con risoluzione al nanosecondo.

Topic ROS 2 e Messaggi I dati sensoriali vengono pubblicati sui topic standard riportati nella Tabella 2.7:

Tabella 2.7: Topic ROS 2 per le Unità Inerziali (IMU) e il Sensore di Luminosità.

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/accel_left/imu</code>	<code>sensor_msgs/msg/Imu</code>	Accelerazione lineare IMU sinistra (headset).
<code>/magic_leap2/gyro_left/imu</code>	<code>sensor_msgs/msg/Imu</code>	Velocità angolare IMU sinistra (headset).
<code>/magic_leap2/accel_right/imu</code>	<code>sensor_msgs/msg/Imu</code>	Accelerazione lineare IMU destra (headset).
<code>/magic_leap2/gyro_right/imu</code>	<code>sensor_msgs/msg/Imu</code>	Velocità angolare IMU destra (headset).
<code>/magic_leap2/accel_compute_pack/imu</code>	<code>sensor_msgs/msg/Imu</code>	Accelerazione lineare IMU Compute Pack.
<code>/magic_leap2/gyro_compute_pack/imu</code>	<code>sensor_msgs/msg/Imu</code>	Velocità angolare IMU Compute Pack.
<code>/magic_leap2/light_sensor</code>	<code>sensor_msgs/msg/Illuminance</code>	Livello di illuminamento ambientale misurato.

Limitazioni ed Eventuali Criticità La frequenza di campionamento esposta dallo strato Android per le unità inerziali risulta vincolata a circa 40 Hz. Tuttavia, il flusso si mantiene costante e privo di jitter rilevante, garantendo una sincronizzazione temporale precisa con i flussi visivi.

2.4 Manuale D'uso

In questa sezione viene illustrata la procedura operativa per l'installazione, la configurazione e l'avvio del sistema a partire dal repository del progetto.

2.4.1 Configurazione dell'Ambiente ROS 2

Sul computer host è necessario predisporre il workspace ROS 2:

1. **Importazione dei Package:** All'interno della cartella `src/` del workspace, clonare il package `config_ml2_stream` (gestione configurazione) e il package `ros_tcp_endpoint` (gestione socket TCP).
2. **Personalizzazione del File `config.yaml`:** Modificare i flag e i parametri nel file `config.yaml` secondo le esigenze dell'esperimento.
3. **Compilazione del Workspace:** Compilare ed effettuare il sourcing dell'ambiente:

```
cd ~/ros2_ws
colcon build --packages-select ros_tcp_endpoint config_ml2_stream
source install/setup.bash
```

2.4.2 Installazione e Configurazione dell'Applicazione su Magic Leap 2

L'applicazione può essere installata senza dover ricompilare il progetto da Unity:

1. **Installazione dell'APK:** Collegare il visore via USB-C e installare il file `.apk` tramite la suite Magic Leap Hub 3.
2. **Requisiti di Rete:** Configurare la workstation con indirizzo IP statico `192.168.0.203` (porta 10000) e verificare che entrambi i dispositivi siano raggiungibili sulla medesima sottorete locale.
3. **Concessione dei Permessi di Sistema:** Prima dell'avvio, accedere dal visore a *Settings* → *Apps* → scheda *Permissions* e abilitare esplicitamente le autorizzazioni per *Camera*, *Spatial Mapping/Perception*, *Sensors* e *Local Network*.

2.4.3 Sequenza di Avvio e Streaming dei Dati

Per garantire il corretto handshake iniziale, rispettare il seguente ordine di avvio:

1. **Avvio dell'Endpoint TCP** (repository GitHub)

```
ros2 launch ros_tcp_endpoint endpoint.py
```

2. **Avvio del Nodo di Configurazione**(codice sorgente)

```
ros2 launch config_ml2_stream magic_leap_config.launch.py
```

3. **Esecuzione dell'Applicazione su Visore:** Avviare l'applicazione dal visore Magic Leap 2; il client si conatterà al server, invierà la richiesta su `/request`, effettuerà il parsing del file JSON ricevuto e attiverà i flussi richiesti.

2.5 Modularità e Upgrade

2.5.1 Setup dell'Ambiente di Sviluppo Unity

Qualora sia necessario ricompilare o estendere l'applicazione:

1. **Creazione del Progetto Unity:** Creare un progetto basato su Unity LTS con supporto Magic Leap 2 OpenXR.
2. **Importazione degli Script:** Associare a un GameObject centrale tutti gli script C# del modulo.
3. **Integrazione del Plugin Android Nativo:** Copiare il file .aar in Assets/Plugins/Android, e inserire nel file mainTemplate.gradle la dipendenza:

```
dependencies {  
    implementation "org.jetbrains.kotlin:kotlin-stdlib:1.9.20"  
}
```

4. **Installazione del Package ROS-TCP-Connector:** Installare il pacchetto tramite Git URL nel Package Manager di Unity e selezionare ROS2 nella scheda di configurazione delle preferenze di rete.

2.5.2 Linee Guida per l'Estensione e l'Upgrade del Codice

Per integrare ulteriori funzionalità o sensori:

1. **Estensione della Configurazione (Configurator.cs):** Aggiungere i nuovi campi alla classe AppConfig per consentirne la valorizzazione automatica dal file config.yaml.
2. **Propagazione Parametri:** Nello script coordinatore, assegnare i parametri letti ai rispettivi componenti prima di abilitarne il flag di avvio.
3. **Sviluppo di Nuovi Moduli:** Creare script indipendenti che gestiscano il campionamento, espongano un flag booleano di attivazione e serializzino i dati verso il connettore ROS 2.

Capitolo 3

Estensione del Bridge: Robot Interaction

3.1 Obiettivo e Architettura dell'Applicazione di Interazione

L'infrastruttura di streaming descritta nei capitoli precedenti fornisce un canale dati bidirezionale ad alte prestazioni. L'estensione applicativa presentata in questo capitolo sfrutta tale ponte software per realizzare un'interfaccia di teleoperazione e guida intuitiva in Realtà Aumentata (AR) per un manipolatore collaborativo a 7 gradi di libertà **Franka Emika Panda**.

L'applicazione trasforma il visore Magic Leap 2 in uno strumento di input spaziale 3D diretto. L'operatore può indicare posizioni target sul piano di lavoro mediante tecniche di puntamento ottico (*raycasting*) con il controller 6-DoF, visualizzare istantaneamente un riscontro olografico sul punto selezionato e demandare al framework di motion planning **MoveIt 2** il calcolo e l'esecuzione di traiettorie prive di collisioni.

Figura 3.1: Flusso operativo dell'estensione: targetting tramite raycasting in Unity, pubblicazione del messaggio PoseStamped e pianificazione della cinematica inversa su MoveIt 2 per il controllo del cobot.

Il flusso operativo complessivo si articola in tre macro-fasi sequenziali:

1. **Localizzazione e Calibrazione Spaziale (TF2):** Stima della posa reciproca tra il sistema di riferimento del visore e la cella di lavoro mediante l'ancoraggio ottico a un marcatore AprilTag.
2. **Selezione del Target in Realtà Aumentata:** Calcolo dell'intersezione 3D tra il raggio emesso dal controller e un piano virtuale calibrato, con rendering visivo di una sfera olografica sul target.
3. **Pianificazione ed Esecuzione Cinematica:** Ricezione della posa target sul topic ROS 2 dedicato, conversione nel frame base del robot e generazione delle traiettorie dei giunti con MoveIt 2.

3.2 Localizzazione Spaziale e Calibrazione tramite AprilTag

All'avvio dell'applicazione su Magic Leap 2, il sistema operativo istanzia un sistema di coordinate radice arbitrario denominato `magicleap_world`. Poiché tale terna di riferimento dipende dalla posizione iniziale dell'operatore nello spazio fisico, non è possibile inviare comandi direttamente al robot senza una procedura di calibrazione che colleghi il mondo virtuale alla cinematica del manipolatore.

3.2.1 Albero delle Trasformazioni Spaziali (TF Tree)

Per unificare i frame di coordinate viene impiegata la libreria standard **TF2** di ROS 2. Ciascuna telecamera del visore pubblica a 60 Hz sul topic standard `/tf` la propria posa variabile rispetto al frame globale `magicleap_world`.

Attraverso un nodo dedicato basato sul package `apriltag_ros`, le immagini acquisite dalla telecamera RGB centrale (*Picture Center*) vengono processate a runtime per individuare un marcatore planare appartenente alla famiglia **tag36h11** posizionato sul tavolo di lavoro.

3.2.2 Composizione delle Trasformazioni

Il nodo di tracciamento individua la posa del marcatore rispetto al frame ottico della telecamera, rappresentata dalla matrice di trasformazione omogenea T_{camera}^{tag} . Attraverso un nodo dedicato si interroga il buffer temporale di TF2 in modo che il sistema risalga alla trasformata della telecamera rispetto all'origine del visore $T_{magicleap_world}^{camera}$.

La posa globale fissa del tag viene calcolata istante per istante mediante la composizione di catene di trasformazioni:

$$T_{tag}^{magicleap_world} = T_{camera}^{magicleap_world} \cdot T_{tag}^{camera} \quad (3.1)$$

Tale trasformata viene pubblicata a 20 Hz sul topic `/magicleap_to_tag4_pose`, fornendo ad Unity un'ancora spaziale solida. L'applicazione grafica posiziona stabilmente un GameObject di riferimento sul marker, che rappresenta il piano di lavoro su cui operare insieme al braccio robotico.

```
1  #!/usr/bin/env python3
2  import rclpy
3  from rclpy.node import Node
4  from geometry_msgs.msg import PoseStamped
5  from tf2_ros import TransformException
6  from tf2_ros.buffer import Buffer
7  from tf2_ros.transform_listener import TransformListener
8
9  class TFChainLinker(Node):
10
11     def __init__(self):
12         super().__init__('tf_chain_linker',
13             automatically_declare_parameters_from_overrides=False)
14
15         # 1. Inizializziamo il Buffer e il Listener delle TF
16         self.tf_buffer = Buffer()
```

```

16         self.tf_listener = TransformListener(self.tf_buffer, self)
17
18         # 2. Creiamo un publisher per inviare la posa finale
19         combinata
20         self.publisher_ = self.create_publisher(PoseStamped, '/
21         magic Leap_to_tag4_pose', 10)
22
23         # 3. Timer per calcolare la trasformata a 20Hz (ogni 0.05
24         secondi)
25         self.timer = self.create_timer(0.05, self.timer_callback)
26
27         self.get_logger().info("Nodo di calcolo TF avviato. In
28         ascolto della catena...")
29
30     def timer_callback(self):
31         # Definiamo il frame di partenza (Padre) e quello di arrivo
32         (Figlio)
33         target_frame = 'tag4'
34         source_frame = 'magic Leap_world'
35
36         if not self.tf_buffer.can_transform(target_frame,
37         source_frame, rclpy.time.Time(), rclpy.duration.Duration(seconds
38         =0.1)):
39             return
40
41         try:
42             # Chiediamo alla TF di unire i punti e calcolare la
43             trasformata diretta
44             now = rclpy.time.Time()
45             trans = self.tf_buffer.lookup_transform(
46                 source_frame,
47                 target_frame,
48                 now)
49
50             # Costruiamo il messaggio PoseStamped da inviare a
51             Unity
52             msg = PoseStamped()
53             msg.header.stamp = self.get_clock().now().to_msg()
54             msg.header.frame_id = source_frame # magic Leap_world
55
56             # Assegniamo la posizione risultante dal calcolo della
57             TF
58             msg.pose.position.x = trans.transform.translation.x
59             msg.pose.position.y = trans.transform.translation.y
60             msg.pose.position.z = trans.transform.translation.z
61
62             # Assegniamo la rotazione risultante dal calcolo della
63             TF
64             msg.pose.orientation.x = trans.transform.rotation.x
65             msg.pose.orientation.y = trans.transform.rotation.y
66             msg.pose.orientation.z = trans.transform.rotation.z
67             msg.pose.orientation.w = trans.transform.rotation.w
68
69             # Pubblichiamo la posa finale
70             self.publisher_.publish(msg)
71
72     except TransformException as ex:
73         # Se la TF non e' ancora pronta, evita il crash e

```

```

63     avvisa self.get_logger().warn(f'Impossibile calcolare la TF: {
        ex}')
64
65 def main(args=None):
66     rclpy.init(args=args)
67     node = TFChainLinker()
68     try:
69         rclpy.spin(node)
70     except KeyboardInterrupt:
71         pass
72     finally:
73         node.destroy_node()
74         rply_shutdown = rclpy.shutdown()
75
76 if __name__ == '__main__':
77     main()

```

Listing 3.1: Nodo ROS 2 `tf_transformation_node.py` per la composizione e pubblicazione della posa del tag rispetto al frame globale.

3.3 Interazione 3D e Definizione del Target in Realtà Aumentata

3.3.1 Piano di Lavoro Virtuale e Raycasting da Controller

All'interno della scena Unity viene istanziato un piano geometrico virtuale con collider, gerarchicamente ancorato al frame dell'AprilTag precedentemente allineato. L'operatore impugna il controller 6-DoF del Magic Leap 2, il quale proietta un raggio laser virtuale (*raycast*) nello spazio tridimensionale.

Ad ogni ciclo di esecuzione:

- Il motore fisico di Unity calcola il punto di impatto (x, y, z) del raggio con il piano di lavoro virtuale.
- Sul punto di intersezione viene renderizzato un oggetto olografico (una sfera 3D luminosa di colore verde) che segue in tempo reale il puntamento dell'utente, fornendo un feedback visivo immediato della coordinata selezionata.
- Alla pressione del trigger sul controller, la coordinata (x, y, z) e l'orientamento di approccio desiderato vengono campionati e preparati per la trasmissione a ROS 2.

3.3.2 Struttura del Messaggio di Comunicazione

La posa del punto selezionato viene incapsulata in una struttura conforme al tipo standard `geometry_msgs/msg/PoseStamped` e trasmessa attraverso il socket TCP verso il nodo server.

Il messaggio è pubblicato sul topic dedicato descritto nella Tabella 3.1:

Tabella 3.1: Interfaccia Topic ROS 2 per il controllo del target del manipolatore.

Topic	Tipo di Messaggio	Descrizione
/target_ball_pose	geometry_msgs/ msg/PoseStamped	Coordinate tridimensionali e orientamento target selezionati via AR.

I campi fondamentali del payload includono:

- `header.frame_id = "magicleap_world"`: specifica che la terna di riferimento per le coordinate è quella globale del visore;
- `header.stamp`: marca temporale di generazione del comando;
- `pose.position`: vettore $[x, y, z]$ indicante la traslazione metrica sul piano di lavoro;
- `pose.orientation`: quaternione unitario $[q_x, q_y, q_z, q_w]$ rappresentante l'orientamento richiesto per l'end-effector.

3.4 Pianificazione del Moto ed Esecuzione con MoveIt 2

Sul PC di controllo del robot è attivo un nodo C++ integrato con il framework **MoveIt 2**, sottoscritto al topic `/target_ball_pose`.

3.4.1 Conversione delle Coordinate nel Frame Base del Robot

Alla ricezione del messaggio `PoseStamped`, il nodo non può inviare direttamente le coordinate all'attuatore poiché espresse rispetto a `magicleap_world`. Tramite un listener TF2, la posa target P^{ML} viene trasformata nel sistema di riferimento locale alla base del manipolatore Franka Emika (`panda_link0`) applicando la trasformazione geometrica nota $T_{panda_link0}^{magicleap_world}$ ricavata dalla catena di calibrazione dell'AprilTag:

$$P^{panda} = T_{panda_link0}^{magicleap_world} \cdot P^{ML} \quad (3.2)$$

3.4.2 Cinematica Inversa e Generazione delle Traiettorie

La posa P^{panda} viene impostata come obiettivo cinematico per l'end-effector all'interno dell'interfaccia `MoveGroupInterface` di **MoveIt 2**. Il solver cinematico esegue:

1. **Risoluzione della Cinematica Inversa (IK)**: Calcolo dei valori dei 7 angoli di giunto del robot per raggiungere la posa cartesiana richiesta.
2. **Motion Planning e Collision Checking**: Pianificazione di una traiettoria fluida nello spazio dei giunti tramite algoritmi OMPL (es. RRT-Connect), validando l'assenza di auto-collisioni e interferenze con il piano di lavoro.

3. **Controllo in Feedback ed Esecuzione:** Invio dei setpoint di posizione e velocità ai controller hardware del cobot tramite l'interfaccia *ros2_control*.

3.5 Procedura di Avvio e Configurazione Sperimentale

Per avviare la pipeline completa di interazione robotica è necessario predisporre l'infrastruttura ROS 2 sulla workstation e lanciare i nodi nel corretto ordine logico:

1. **Avvio dell'Endpoint TCP:** Inizializza il server socket sulla porta 10000 (repository GitHub):

```
ros2 launch ros_tcp_endpoint endpoint.py
```

2. **Avvio del Nodo di Rilevamento AprilTag:** Attiva l'elaborazione del flusso video per il tracking del marcatore 36h11 (repository GitHub):

```
ros2 launch apriltag_ros ml2_camera_36h11.launch.py
```

```
1 import os
2 from ament_index_python.packages import
   get_package_share_directory
3 from launch import LaunchDescription
4 from launch_ros.actions import Node
5
6 def generate_launch_description():
7     # 1. Recupera il percorso del file di configurazione YAML
   di apriltag_ros
8     apriltag_share_dir = get_package_share_directory('
   apriltag_ros')
9     params_file_path = os.path.join(apriltag_share_dir, 'cfg',
   'tags_36h11.yaml')
10
11     # 2. Definisce il nodo apriltag
12     apriltag_node = Node(
13         package='apriltag_ros',
14         executable='apriltag_node',
15         name='apriltag_node',
16         output='screen',
17         # Parametri esterni + forzatura del trasporto compresso
18         parameters=[
19             params_file_path,
20             {
21                 'image_transport': 'compressed'
22             }
23         ],
24         # Rimappatura dei topic
25         remappings=[
26             ('image_rect', '/magic_leap2/color/image_raw'),
27             ('camera_info', '/magic_leap2/color/camera_info')
28         ]
29     )
30
```

```

31     # 3. Crea la descrizione del lancio e vi aggiunge il nodo
32     return LaunchDescription([
33         apriltag_node
34     ])

```

Listing 3.2: File di lancio ROS 2 `ml2_camera_36h11.launch.py` per l'inizializzazione del nodo `apriltag_ros` con trasporto compresso e rimappatura dei topic.

- 3. Avvio del Nodo di Configurazione:** Gestisce i parametri di configurazione per lo streaming di sensori e camere del visore (codice sorgente):

```
ros2 launch config_ml2_stream magic_leap_config.launch.py
```

```

1 /config_ML_node:
2   ros__parameters:
3
4     general:
5       topic_name: "/magic_leap2"
6
7     picture_center:
8       enabled: true
9       # stream 0 is the simple RGB camera stream.
10      # stream 1 is the AR stream with the world as background
11      and the user as foreground.
12      stream: 0
13      # for stream 0 the possible resolution are 640x480, 1280
14      x720, 1920x1080, 3840x2160, 2048x1536, 1280x960, 1440x1080,
15      2880x2160, 4096x3072.
16      # for stream 1 the possible resolution are 648x720, 972
17      x1080, 1944x2160, 960x720, 1440x1080, 2880x2160.
18      resolution_width: 1920
19      resolution_height: 1080
20      update_rate: 30 # possible values in
21      range [30, 60]
22      format: "jpeg"
23
24     depth_center:
25       enabled: false
26       # stream 0 is the long range depth stream.
27       # stream 1 is the short range depth stream.
28       stream: 0
29       resolution_width: 544
30       resolution_height: 480
31       # for stream 0 the possible update rates are 1, 5.
32       # for stream 1 the possible update rates are 5, 30, 60.
33       update_rate: 5
34       format: "Depth32" # possible values = "Depth32", "
35       DepthRaw".
36
37     world_center:
38       enabled: false
39       stream: 0 # possible values
40       = 0, 1
41       resolution_width: 1016
42       resolution_height: 1016
43       update_rate: 30
44       format: "Grayscale"

```

```

38
39     world_right:
40         enabled: false
41         stream: 0                                # possible values
42         = 0, 1
43         resolution_width: 1016
44         resolution_height: 1016
45         update_rate: 30
46         format: "Grayscale"
47
48     world_left:
49         enabled: false
50         stream: 0                                # possible values
51         = 0, 1
52         resolution_width: 1016
53         resolution_height: 1016
54         update_rate: 30
55         format: "Grayscale"
56
57     eye_temple_right:
58         enabled: false
59         stream: 0
60         resolution_width: 400
61         resolution_height: 400
62         update_rate: 30
63         format: "Grayscale"
64
65     eye_temple_left:
66         enabled: false
67         stream: 0
68         resolution_width: 400
69         resolution_height: 400
70         update_rate: 30
71         format: "Grayscale"
72
73     eye_nasal_right:
74         enabled: false
75         stream: 0
76         resolution_width: 400
77         resolution_height: 400
78         update_rate: 30
79         format: "Grayscale"
80
81     eye_nasal_left:
82         enabled: false
83         stream: 0
84         resolution_width: 400
85         resolution_height: 400
86         update_rate: 30
87         format: "Grayscale"
88
89     # the other sensors include gyroscope, accelerometer and
90     ambient light sensor.
91     other_sensors:
92         enabled: false

```

Listing 3.3: File di configurazione config.yaml pronto per il corretto utilizzo dell'applicazione.

4. **Avvio del Nodo di Composizione Trasformazioni TF:** Calcola a 20 Hz la posa unificata tra `magicleap_world` e il tag (codice sorgente):

```
ros2 launch robot_interaction_ml2 tf_transformation.launch.py
```

5. **Avvio dell'Ambiente MoveIt 2 e Driver Robot:** Inizializza il modello cinematico
6. **Esecuzione dell'App su Magic Leap 2:** L'avvio dell'applicazione su visore completa l'handshake e attiva il raycasting interattivo con rendering olografico.

3.6 Fasi di Esecuzione

Al fine di validare l'intera catena di comunicazione e controllo, il sistema è stato testato sperimentalmente nella cella robotica integrando il visore Magic Leap 2 con il cobot Franka Emika Panda. La sequenza operativa si articola in tre momenti sequenziali descritti di seguito.

3.6.1 Puntamento 3D e Selezione del Target

L'operatore inquadra l'area di lavoro ed emette un raggio virtuale (*raycasting*) tramite il controller a 6 gradi di libertà. L'intersezione con il piano calibrato mediante AprilTag genera una sfera verde olografica che fornisce un riscontro visivo istantaneo della coordinata selezionata. Alla conferma dell'utente, la posa calcolata viene inviata a ROS 2 sul topic `/target_ball_pose`.

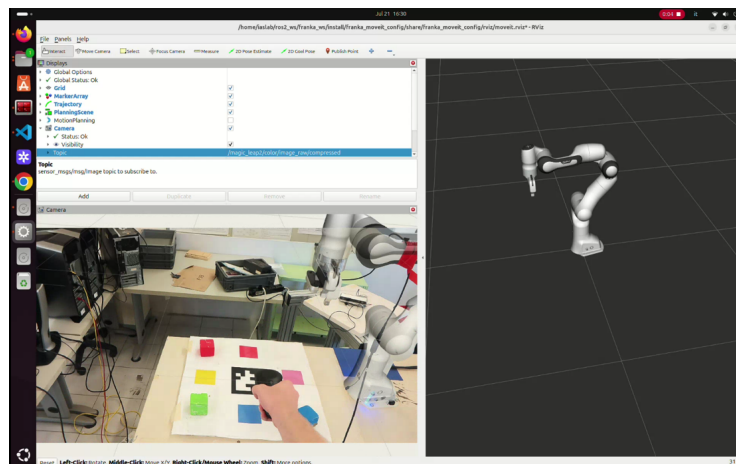


Figura 3.2: Fase 1: Selezione della posa target sul piano calibrato tramite raycasting e visualizzazione del riscontro olografico nel visore.

3.6.2 Pianificazione ed Esecuzione della Traiettoria

Il framework **MoveIt 2** riceve il messaggio contenente la coordinata, la converte nel sistema di riferimento della base del manipolatore (`panda_link0`) e calcola la cinematica inversa per pianificare una traiettoria priva di collisioni. Il cobot **Franka Emika Panda** esegue il moto cartesiano, raggiungendo con precisione la posizione target impostata in AR.

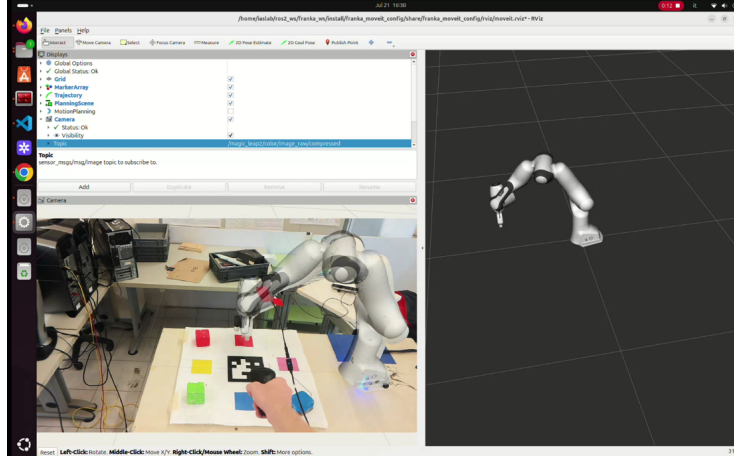


Figura 3.3: Fase 2: Esecuzione del moto cartesiano e raggiungimento del punto target da parte del braccio manipolatore Franka Emika Panda.

3.6.3 Ritorno in Configurazione di Sicurezza

Completato il compito sul target, il robot calcola ed esegue automaticamente una traiettoria di rientro verso la configurazione di riposo (*Home Pose*). Questo movimento libera l'area di lavoro condivisa per tutelare l'incolumità dell'operatore, nel rispetto dei principi di *Human-Robot Collaboration* (HRC), e predispone il sistema alla ricezione del comando successivo.

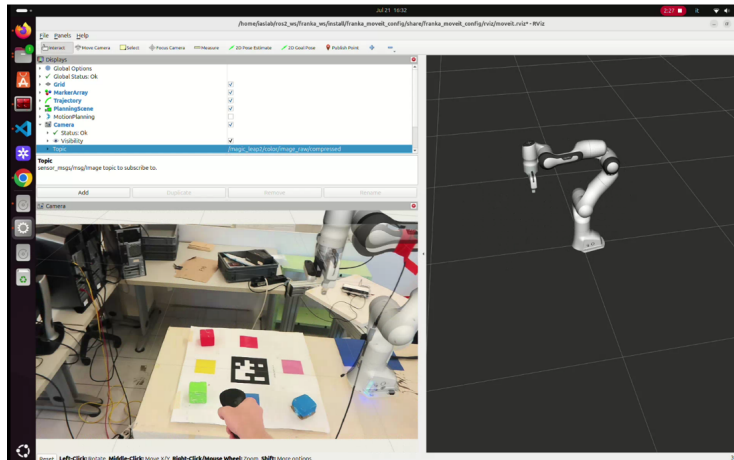


Figura 3.4: Fase 3: Rientro automatico del manipolatore nella posizione di sicurezza (*Home Pose*) per il disimpegno dell'area operativa.

3.7 Validazione Sperimentale e Risultati

La validazione sperimentale ha confermato l'efficacia e la stabilità dell'architettura in condizioni operative reali:

- **Streaming Video e Latenza di Controllo:** I frame video e i metadati viaggiano attraverso la rete senza jitter rilevante o blocchi di trasmissione, permettendo un controllo real-time.

- **Accuratezza dell'Allineamento Spaziale:** La combinazione tra il tracciamento nativo a 6-DoF del Magic Leap 2 e il nodo AprilTag ha garantito la perfetta stabilità degli ologrammi, mantenendo la sfera target saldamente ancorata al piano reale senza deriva percettibile anche in presenza di ampi spostamenti fisici dell'utente.
- **Intuitività ed Ergonomia HRC:** La guida tramite puntamento 3D elimina la complessità delle interfacce convenzionali su monitor 2D, permettendo una cooperazione uomo-robot rapida, naturale e conforme ai requisiti di sicurezza.